

Process synchronization

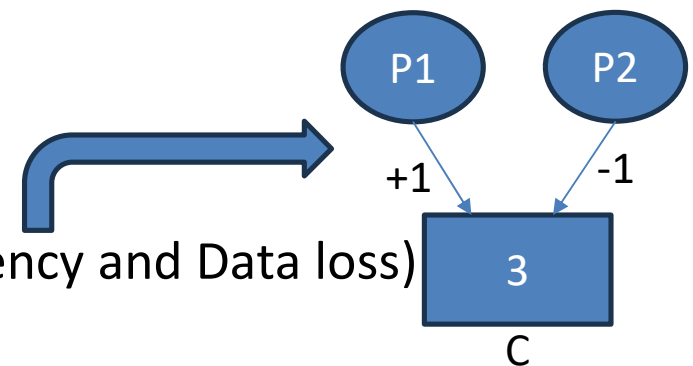
- What is inter process communication (IPC)? : Can be done through files or by function through global variables

1. # Problems due to lack of synchronization→

- Inconsistency
- Data loss
- Deadlock

2. # Type of synchronization→

- Competitive → Prob: (Inconsistency and Data loss)
- co-operative → Prob: (Deadlock)



Note: An application in an IPC environment may involve either or both type of synchronization.

- ❖ Two or more processes are said to be in **competition** iff they compete for the accessibility of shared resource.
- ❖ Here Processes 1 and 2 tries to update the value of a shared variable 'C' at the same time.
- ❑ Two or more processes are said to be in **cooperation** iff they get affected by each other i.e. the execution of one process affect the execution of other process. **Exp: Producer-Consumer Problem.**
- ❑ **Lack of cooperative synchronization among processes may lead to the problem of DEADLOCK.**

- **Illustration of the problem (Producer and Consumer):**

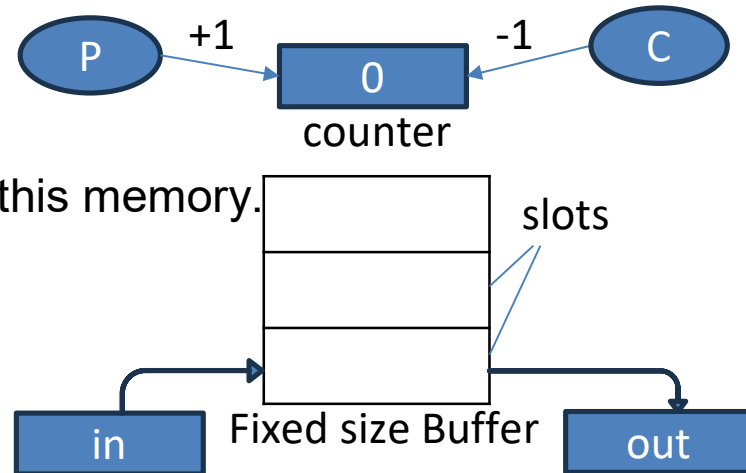
Suppose that we wanted to provide a solution to the producer-consumer problem that fills all the buffers.

- We can do so by having an integer counter that keeps track of the number of full buffers. Initially, counter is set to 0.
- It is incremented by the producer after it produces a new buffer and is decremented by the consumer after it consumes a buffer.

- Processes can execute concurrently.
- May be interrupted at any time, partially completing execution.
- Concurrent access to shared data may result in data inconsistency.
- Maintaining data consistency requires mechanisms to ensure the orderly execution of cooperating processes

Producer and Consumer Problem:

Both processes access this memory.



In operating System Producer is a process which is able to produce data/item.

```
#define N 100           /Global Declarations
Int Buffer [N];         /*shared by P&C
Int count =0;           /*shared by P&C
```

```
Void producer (void)
```

```
{
    int temp, in=0;
    While (True)
    {
        /* produce an item in temp */
        produce item (item p);
        while (count == N);
        buffer[in] = temp;
        in = (in + 1)%N
        count= count+1;
    }
}
```

In operating System consumer is a process which is able to consume data/item.

```
Void consumer (void)
```

```
{
    int temp, out=0;
    While (True)
    {
        while (count == 0) ;
        item c = Buffer [out];
        out = (out + 1)%N
        count= count-1;
        Process item [item c]
    }
}
```

Co-operation

Competition

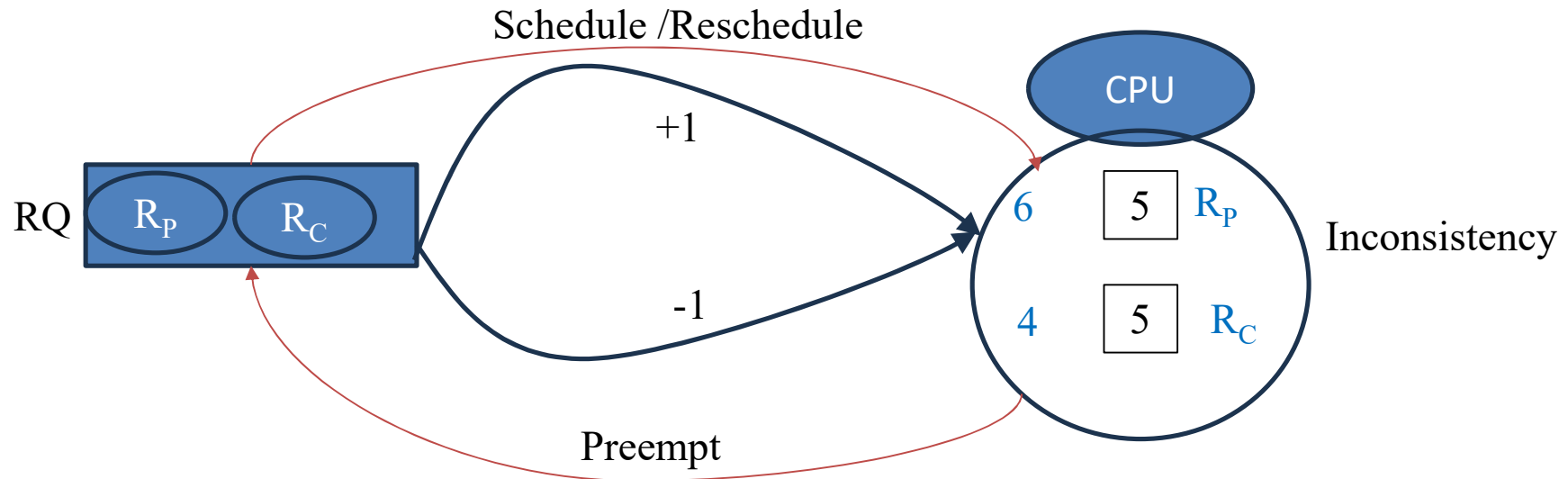
IN indicates the location where the item suppose to be placed in NEXT. "IN" is locating already next position. So, this sequence is right to place items.

SETUP:

- I. Load R_p , $M[\text{count}]$
- II. Inc, R_p
- III. $SM[\text{Count}]$, R_p

- I. Load R_C , $M[\text{count}]$
- II. Dcr, R_C
- III. $SM[\text{Count}]$, R_C

$T_i \rightarrow R_p$: I, II, Preempt
 $T_j \rightarrow R_C$: I, II, III
 $T_k \rightarrow R_p$: III

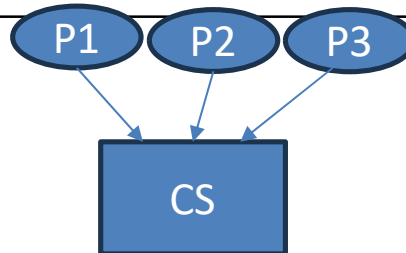


Producer Consumer Problem

- Producer-Consumer problem is a **classical synchronization problem** in the operating system. With the presence of more than one process and limited resources in the system the synchronization problem arises.
- If one resource is shared between more than one process at the same time then it can lead to data inconsistency.
- In the producer-consumer problem, the producer produces an item and the consumer consumes the item produced by the producer.
- Both Producer and Consumer share a common memory buffer. This buffer is a space of a certain size in the memory of the system which is used for storage. The producer produces the data into the buffer and the consumer consumes the data from the buffer.
- Accessing memory buffer should not be allowed to producer and consumer at the same time.
- first, understand the few terms used in the code:
 - "in" used in a producer code represent the next empty buffer.
 - "out" used in consumer code represent first filled buffer.
 - count keeps the count number of elements in the buffer.

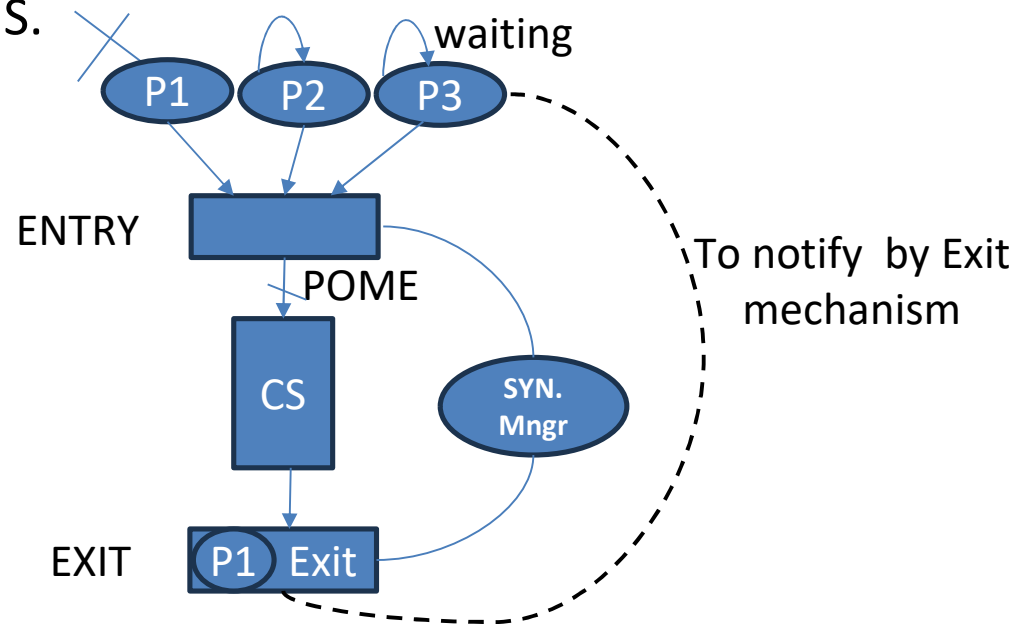
3. # Causes responsible for synchronization problems → (Necessary conditions)

critical section (CS):	part of program where shared resources are accessed. (NonCS: which does not consist shared resources) A program in IPC may involve or may have a series of CS and NonCS
Race condition:	Process must be racing either competitively or cooperatively to access CS, exp: P&C problem, tries to update the count at the same time.
Preemption	

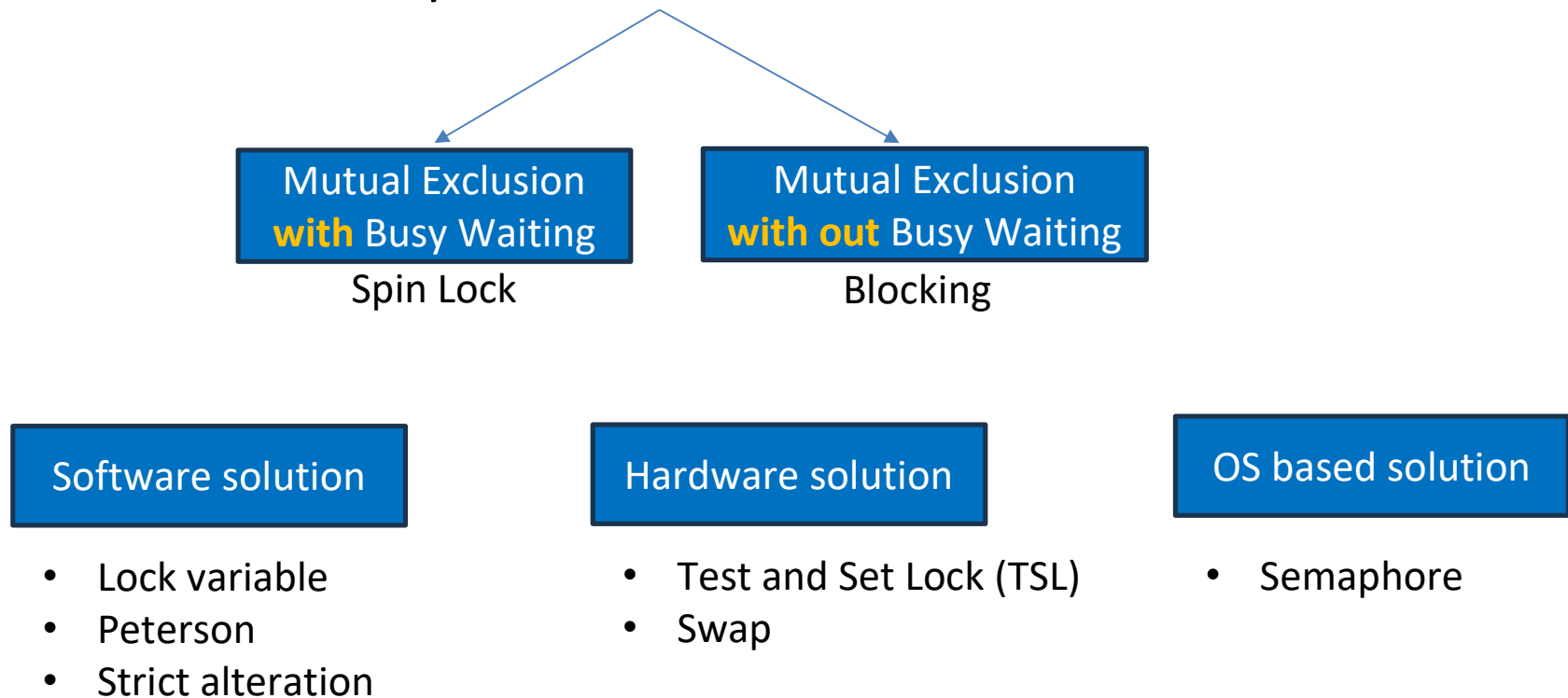


Principal of Mutual Exclusion (POME):

- If a process is present in C.S., other process should not be allowed to enter the C.S. (should be waiting outside the C.S.)
- POME is ensured by syn. Mechanism or synchronization Tool, placed between processes and C.S.



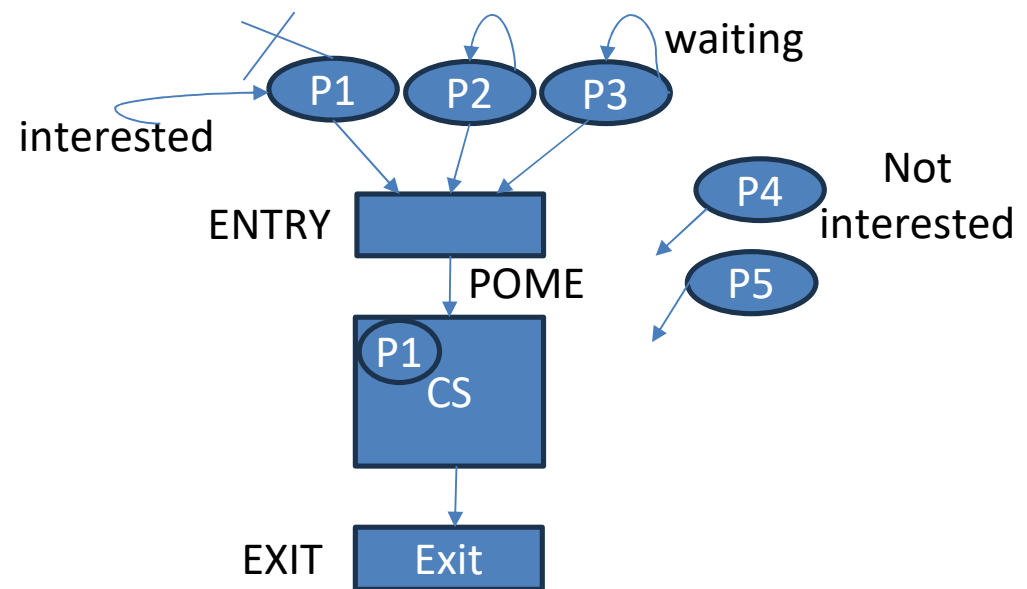
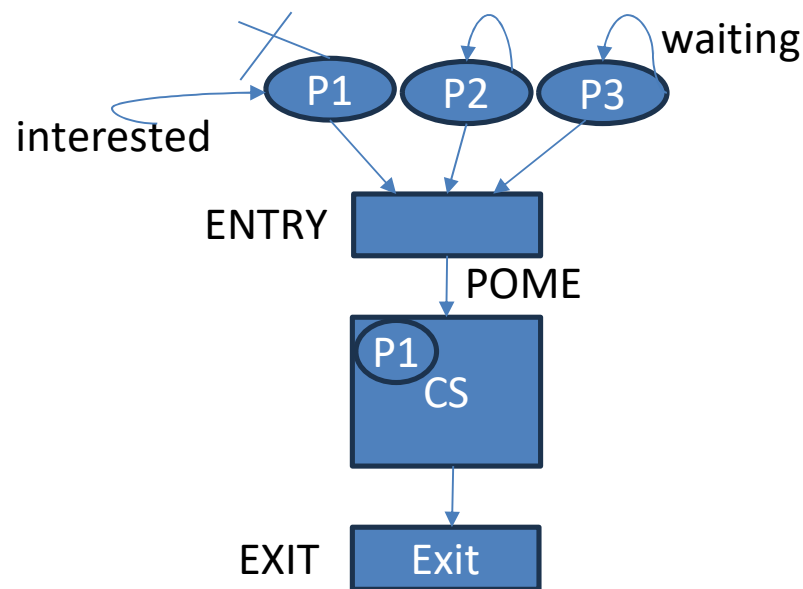
Classification of synchronization Mechanism:



Solution	Processes	Practical
Lock variable	N	
Strict alternation	2	
Peterson's	2	
Mutex/Semaphore	N	More

Requirement of Synchronization Mechanism for solving Critical Section Problem

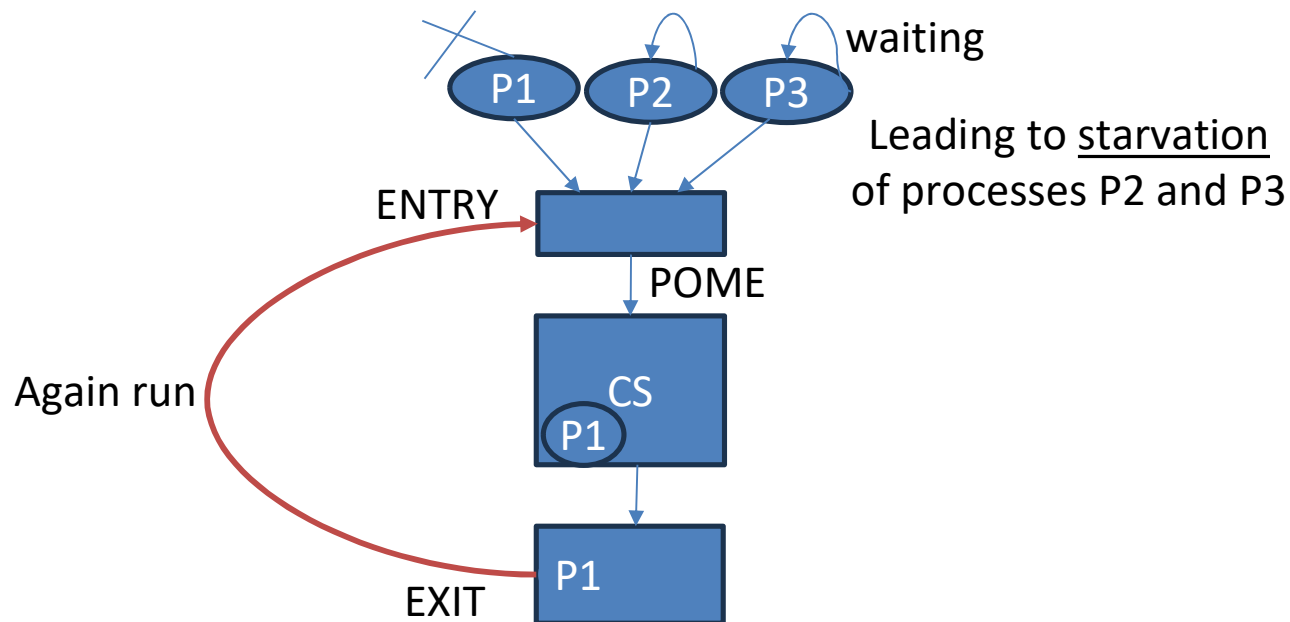
1. Mutual Exclusion: must always be **guaranteed**. //(else inconsistency/data loss)
 - “**All** processes are interested”
2. Progress-
 - “**Some** processes are interested”
 - **Def1**: No process running outside CS should **block** the other interested processes from entering CS.
 - **Def.2**: Processes running outside CS (Non-CS) **should not participate** in decision making of allowing a process to enter CS, among those who are interested.



Cont.

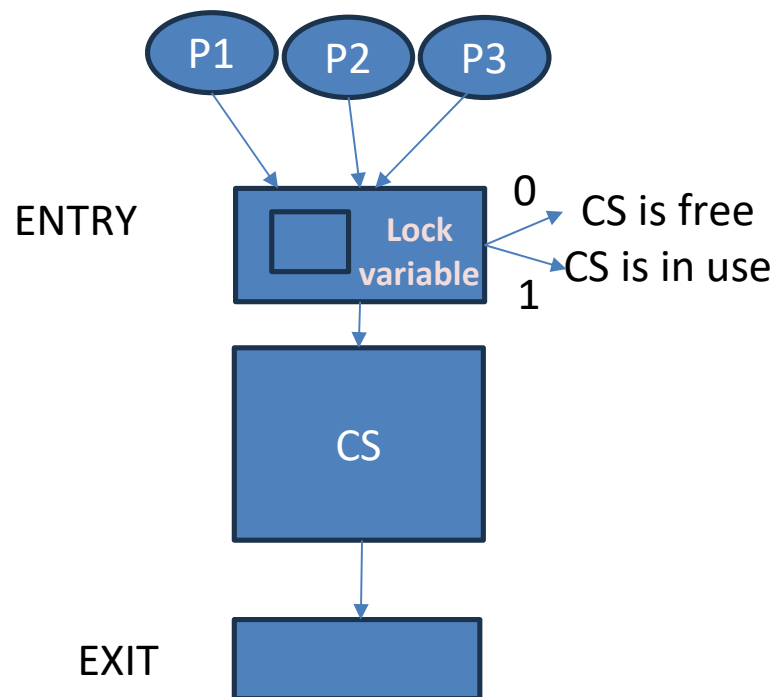
3. Bounded Waiting-

- “All processes are interested”
- **Def1:** No process has to **wait forever** to access its **CS**.
- **Def.2:** There **should be a bound** on number of times a process is allowed to enter CS before the other waiting process's request is granted.



Lock Variable

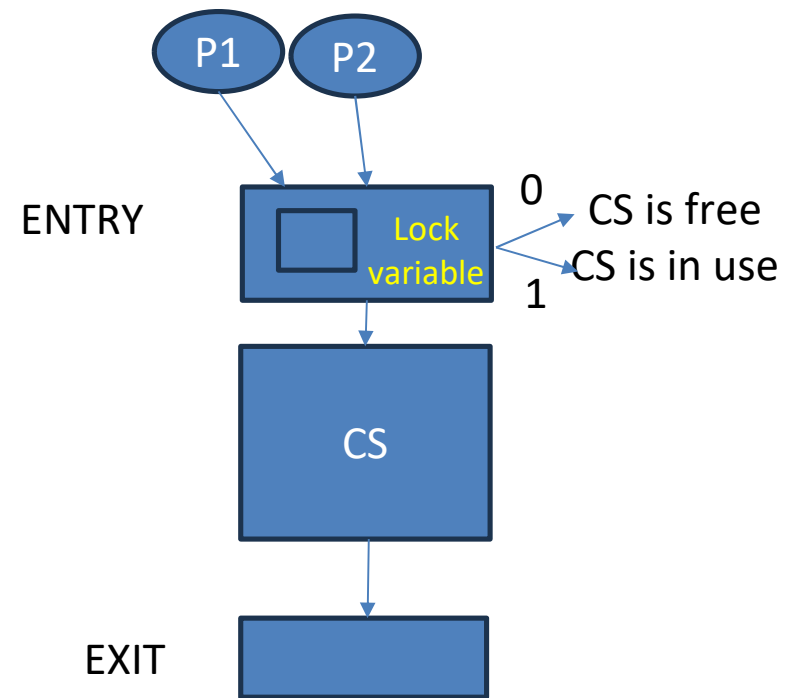
- **ME with Busy waiting &S/W solution**
- **Multi process solution, >2 processes**
- Busy waiting solutions are always implemented using loop mechanism for repeatedly checking.



- `Int Lock =0;`
 - `Void process (int i)`
 - `{`
 - `While (1)`
 - `{`
 - `a) Non-CS ();`
 - `b) While (Lock!=0);`
 - `c) Lock=1`
 - `d) <C.S.>`
 - `e) Lock =0;`
 - `}`
 - `}`
- The diagram also includes a curved arrow labeled 'Busy waiting' pointing from the 'While (Lock!=0);' line back to itself, indicating the loop mechanism.

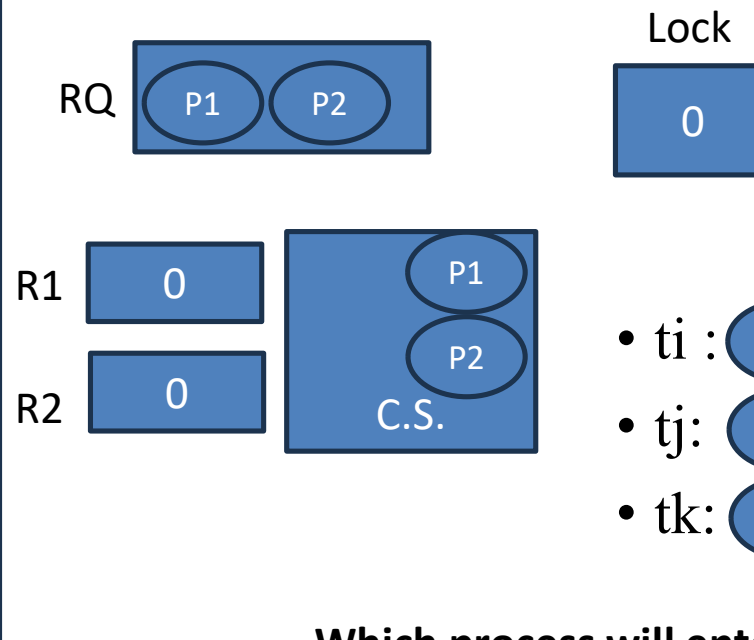
Cont.

- ME guaranteed → no,
only if No preemption
- Progress guaranteed → yes
- BW guaranteed → no (in either case
of Pre/Non-Preemption)



SETUP:

- **I.** load $R_i, M[\text{Lock}]$
- **II.** Compare $R_i, \#0$;
- **III.** Jump IF not Zero, Step I
- **IV.** Store $M[\text{Lock}], \#1$;
- **V.** $\langle \text{CS} \rangle$
- **VI.** Store $M[\text{Lock}], \#0$

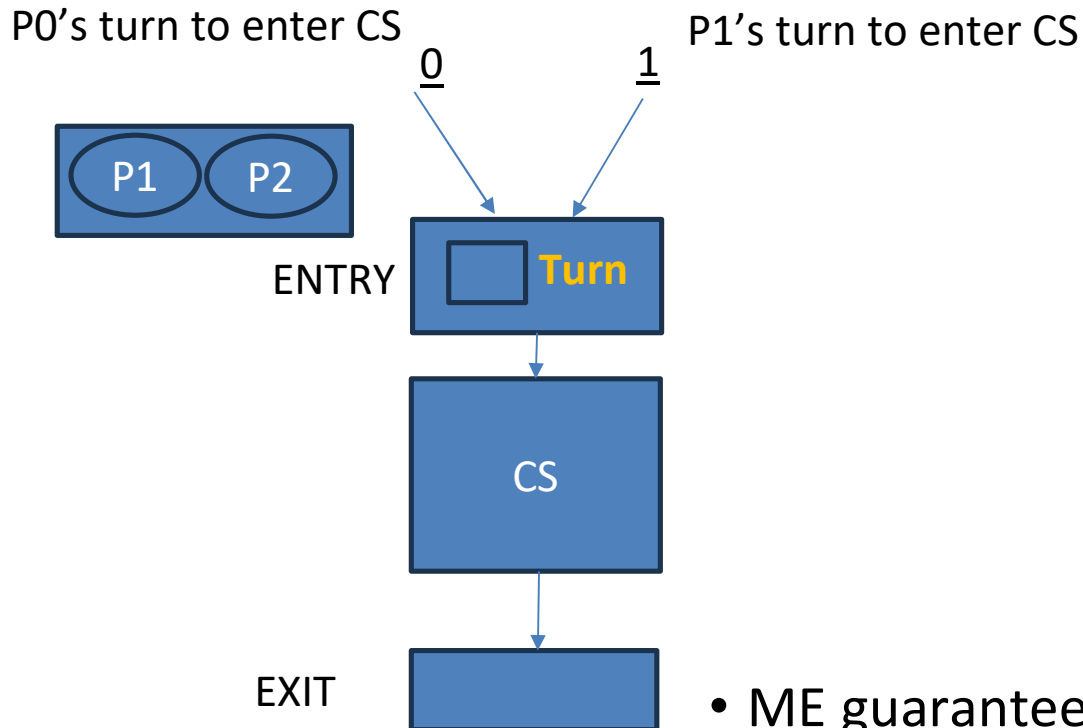


- $t_i : P1 : \text{I, II, III}$
- $t_j : P2 : \text{I, II, III, IV, V}$
- $t_k : P1 : \text{IV, V}$

Which process will enter to CS, first ?

Strict Alteration

- ME with Busy waiting & S/W solution
- 2 process solution (P0, P1; Pi, Pj)
- Strictly on alternate basis, processes takes turn to enter C.S.



• Int turn = rand (0,1);

• Void process (int i)

• {

• While (1)

• {

• a) Non-CS ();

ENTRY

• b) While (turn!=i);

• c) <C.S.>

EXIT

• d) turn =j;

• }

• }

• ME guaranteed → yes

• Progress guaranteed → no

• BW guaranteed → yes

➤ First enter CS then after coming out change the value of ID unlike LOCK variable.

Observations

- It does **not guarantee Progress** due to the fact that a process which is not interested in CS having its ID **set to turn** **prevent the other interested process from accessing CS**.
- In general, an interested process must be allowed to enter CS **anytime** and **any number of times** **as long as CS is free** and other processes are not interested.

Another solution

P0

- While (1)
- {
- flag [0] =T
- While (flag [1]);
- <C.S.>
- flag [0] =F

P1

- While (1)
- {
- flag [1] =T
- While (flag [0]);
- <C.S.>
- flag [1] =F

- ME-Yes
- Progress-No // both interested at same time, else Yes
- BW-No // If both set their flags at the same time
- It will lead to deadlock if both becomes interested at same time.

Peterson's Solution

- # define N 2 //two processes only
 - # define TRUE 1 //define Boolean values
 - # define FALSE 0
 - Int interested [N] = {FALSE}; //shared variable, initially No process want to enter CS
 - Int turn; //shared variable
 - Void process (int i) // i can be 0 or 1
 - {
 - While (1)
 - {
 - a) Non-CS (); // Both processes can execute this simultaneously.
 - b) interested [i] =TRUE; // Process i announces that it wants to enter into the critical section.
 - c) turn =i;
 - d) While (interested [J] == TRUE && turn ==i);// Bust wait condition
 - e) <C.S.>
 - f) interested [i] =FALSE;
 - }
 - }
- ME with Busy waiting &S/W solution
 - 2 process solution (P0, P1; Pi, Pj)
 - Employs advantages of both:
(Lock Variable and Strict Alteration).
- // interested[0] = TRUE → Process 0 wants to enter CS.
// interested[1] = FALSE → Process 1 does not want to enter CS.

- ti :  : a, b
- tj:  : a,b,c,d
- tk:  : c,d //i=0, j=1
- tl:  : d, e
- //i=1, j=0

- ti :  : a, b, c, d, e //considering P1 is not interested
- //P1 is interested
- tj:  : a, b, c, d  Busy wait, ensuring ME
- tk:  : e, f, a, b, c, d

Peterson's Solution

P0

- While (1)
- {
- flag [0] =T
- turn =1
- While (turn ==1 && flag [1]==T);
- <C.S.>
- flag [0] =F

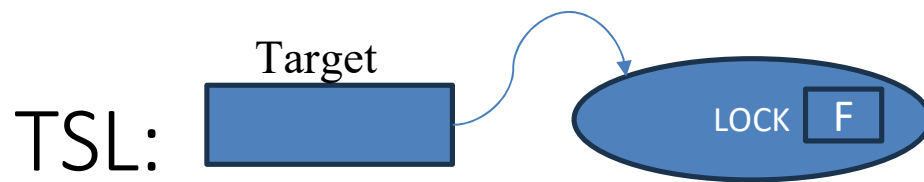
P1

- While (1)
- {
- flag [1] =T
- turn = 0;
- While (turn ==0 && flag [0] ==T);
- <C.S.>
- flag [1] =F

- ME guaranteed →yes
- Progress guaranteed →yes // If P1 is not interested then it is not able to block P0 entering into CS
- BW guaranteed →yes

Test & Set Instructions:-

- In a **Uniprocessor environment** → no concurrent execution,
 - ME can be obtained by preventing process, that is running in Critical Section (CS), from being interrupted. By → disabling the interrupt before process enters into CS.
 - Enable the interrupts only after the process exited from CS.
 - When a process is executing in its CS, it can not be interrupted and thus ME is achieved.
 - However, **in multiprocessor environment**, disabling all interrupts by sending interrupt signal to all processor is time consuming. Therefore, to achieve ME, Test and Set instructions are used.
 - Test and Set instructions are simple H/W instructions that solve the CS problem by providing ME in an easy and efficient way, in Multiprocessor environment.
- The synchronization mechanism is implemented **directly in the CPU hardware**, not in software. **Test & Set** is an **atomic machine instruction** provided by the processor.



- ME with Busy waiting & H/W solution
- Multi-process solution
- Special h/w supported instruction
- **Extended Lock Variable.**

```

• Bool TSL (Bool *Target)
• {
•   Bool r.v.;
•   a) r.v. = *target;
•   b) *Target = TRUE;
•   c) return (r.v.);
• }

```

- TSL returns current value of Lock (pointed by Target) and Sets the value of Lock always TRUE

```

• Bool Lock = FALSE; //GLOBAL variable
• Void process (int i)
• {
•   While (True)
•   {
•       a) Non-CS ();
•       b) While (TSL (&Lock) == True);
•       c) <CS>
•       d) Lock = FALSE;
•   }
• }

```

- Ensures ME
- Ensures Progress
- Not Bounded waiting
- Deadlock free but not starvation free

//process tries to acquire the lock using TSL

- $t_0 : \text{P1} : a, b : \text{TSL}=\text{F}$
- $t_5 : \text{P2} : a, b : \text{TSL}=\text{T}$
- Guarantee ME

TSL:

- **I.** load Ri, M [Lock]
 - **II.** Store M[Lock], #1;
 - **III.** Compare Ri, #0;
 - **IV.** Jump IF not Zero, Step I
 - **V.** <CS>
 - **VI.** Store M[Lock], #0
- **Atomic**, i.e. no preemption between these two statements
- In LOCK variable

LOCK:

- ME guaranteed → no,
only if No preemption
- Progress guaranteed → yes
- BW guaranteed → no (in either case of Pre/Non-Preemption)

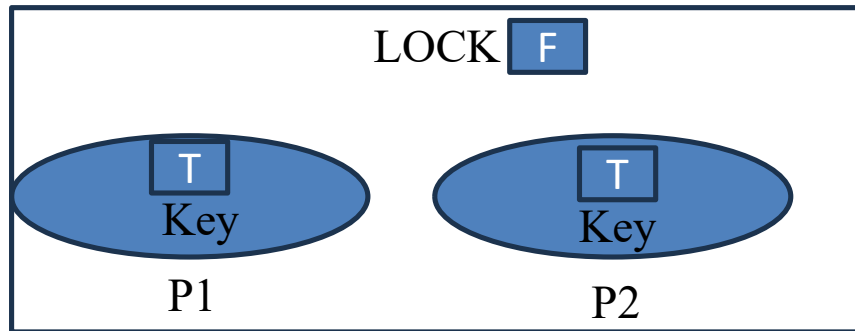
TSL:

- Ensures ME
- Ensures Progress
- Not Bounded waiting

hardware-supported **Swap** instruction

Swap Instruction:

- Void swap (Bool *a, Bool*b)
- {
- Bool t;
- t =*a; temp = key
- *a=*b key = Lock
- *b=t; Lock = temp
- }



- Ensures ME
- Ensures Progress
- Not Bounded waiting

Swap is:

- An **atomic machine instruction**
- Executed directly by the CPU
- Cannot be interrupted in the middle
- This atomicity is provided by **hardware support**, not by the programmer.

- Bool Lock =FALSE; // Shared variable
- void process (int i)
- {
- Bool key; //local variable
- While (1)
- {
- a) Non-CS ();
- b) key=True; //interest
- c) while (key =TRUE)
- {
- Swap (&key, &Lock); //a special CPU instruction
- }
- d) <CS>
- e) Lock =FALSE;
- }
- }

Case 1: Lock is FREE (Lock = FALSE)

- key = TRUE
- Lock = FALSE

After swap:

- key = FALSE
- Lock = TRUE

Now key == FALSE, so loop exits.

Process enters Critical Section.

Case 2: Lock is BUSY (Lock = TRUE)

- key = TRUE
- Lock = TRUE

After swap:

- key = TRUE
- Lock = TRUE

Nothing changes → loop continues.

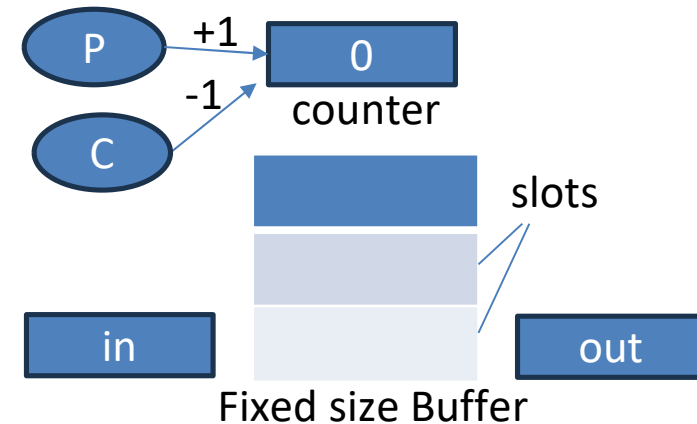
Mutual Exclusion with Blocking/Without Busy wait

1. Sleep ()& Wake up();

- Are OS primitives
- ME with Blocking
- Multi-process Solution

```
Void producer (void)
```

```
{
    int itemp, in=0;
    While (1)
    {
        /* produce an item in temp */
        a) Produce item (itemp);
        b) if(count== N) sleep();
        c) Buffer [in]= itemp;
        d) in = (in + 1) %N
        e) count= count+1;
        f) if (count==1) wakeup(consumer);
    }
}
```



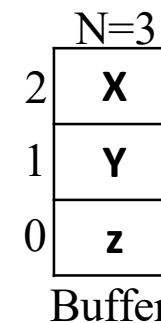
Co-operation

```
{
    a) if(count == 0) sleep ();
    b) itemp = Buffer [out]
    c) out = (out + 1) %N;
    d) count= count-1;
    e) If (count==N-1) wakeup(producer);
    f) process item (item c);
}
```

Competition

- One process wakeups the other
- This code also leading to deadlock

Ready Queue



Sleep Queue



count 0,1,2, 3

- t0 : **C** : Load, compare, Jump, Sleep() //The check and sleep are **not atomic**:
if (count == 0), plan to sleep

Preempt



- t4 : **P** : produces item
if (count == 1), wakeup consumer. But the consumer has not yet gone to sleep, so the wakeup is lost.

- t5 : **C** :sleep();
So, the **consumer sleeps**, even though:
count = 1
(buffer has an item)

- t10 : **P** : after count=N, producer also sleeps.

Why This Happens?

Because **sleep/wakeup is not atomic with the condition check**, causing a **lost wakeup signal**.
This classic problem is why operating systems use **Semaphores**, instead of sleep/wakeup.

Semaphores

- Proposed by Edsger Dijkstra in 1965.//Dutch computer scientist
- M.E. with B.W.
- Semaphore is an O.S. primitive (kernel implementation)
- Software solution at user mode are implemented using semaphore (like system call)
- Semaphore supports two operations:
 - $\rightarrow$ Down/P/Wait
 - $\rightarrow$ Up/V/signal/release
- **P (Proberen)** $\rightarrow$ try to acquire the resource
 - **V (Verhogen)** $\rightarrow$ release the resource
- Semaphore is a variable, it takes only integral values, used in ME by various concurrent cooperative processes to achieve synchronization;
- Two types:
 - $\rightarrow$ Counting semaphore $(-\infty, +\infty)$
 - $\rightarrow$ Binary Semaphore $(0,1)$ $\leftarrow$ **MUTEX**

Counting Semaphores:

- type def
- Struct S
- { Int value;
- Queue type L; } // list of PCBs of those processes that get blocked while performing Down operation, **unsuccessfully**//
- SEMAPHORE;

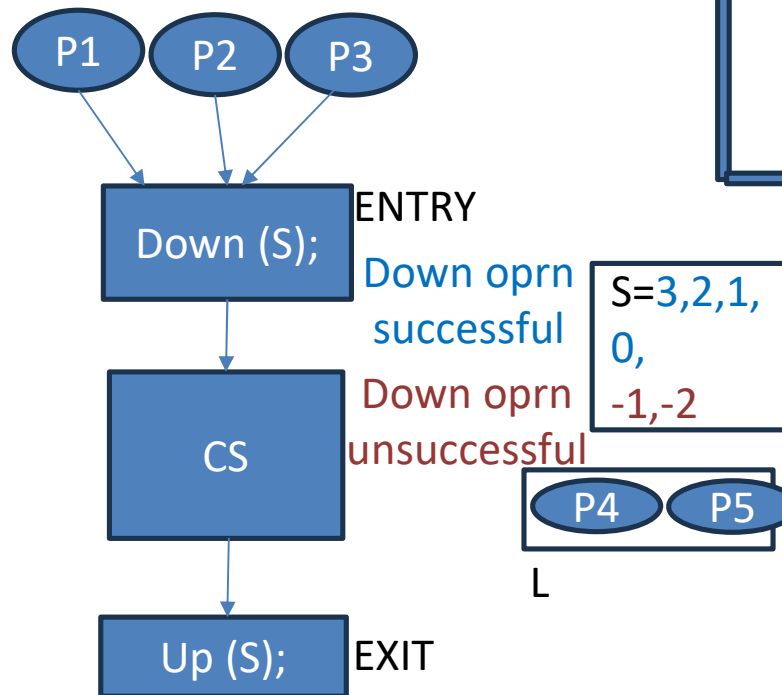
Kernel mode
Atomic
instruction

User mode

- SEMAPHORE S;
- S.value=3;
- -
- -
- Down (s);
- <si> //Shared instruction in CS

- Down (SEMAPHORE S)
- {
- a) S.value=S.value-1;
- b) if (S.value<0)
- { **(unsuccessful)**
Put this process (PCB)
In $S \rightarrow L(Q)$ & block Process (Pi)
&Sleep ()
- }
- else
- return;
- } **(If successful)**

- up (SEMAPHORE S)
- {
- a) S.value=S.value+1;
- b) if (S.value<=0)
- {
Select a process from $S \rightarrow L(Q)$ & wakeup ();
- }
- else
- return;
- }

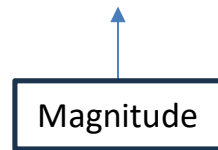


- I. $S = -2; -1 \rightarrow ++ \rightarrow -ve (<0)$
 - II. $S = -1; 0 \rightarrow ++ \rightarrow 0$
 - III. $S = 0; 1 \rightarrow ++ \rightarrow >0 (+ve)$
- Diagram illustrating the state of the semaphore during the Up operation:
- Before Up
 - leftover
 - Up (indicated by a hand icon)

Semaphores

Que: Let $S=4$, then after performing the following operations {10P, 2V, 18P, 3V, 6P, 4V}.

The value of $S = ?$ (-21) representing number of blocked processes.



Binary Semaphores:

```
type def
Struct S
{
enum value =(0,1);
Queue type L;           // list of PCBs of processes
                        that are blocked, while performing
                        Down operation, unsuccessfully
}
BSEMAPHORE;
```

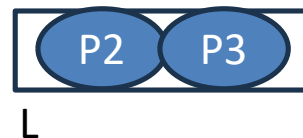
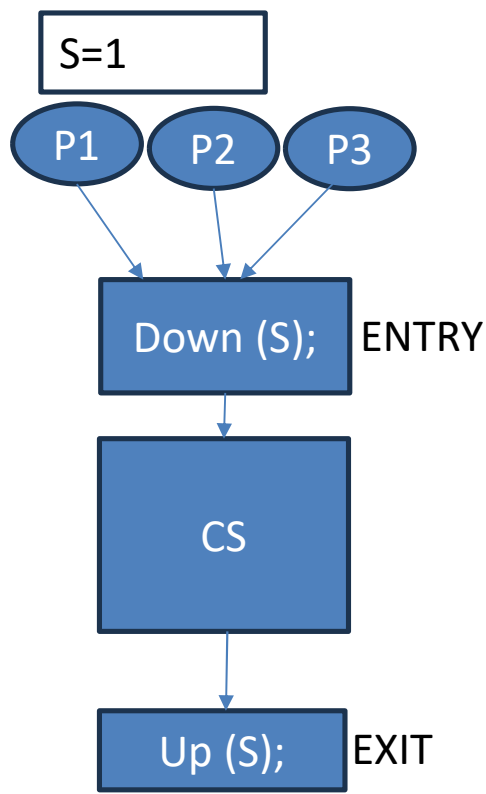
Binary Semaphores

- BSEMAPHORE S; //Declares
- S.value=1;
- -
- -
- Down (s);
- <si> //Shared
- - instruction in CS
- -
- up (S);

- Down (BSEMAPHORE S)
- {
- if (S.value==1)
- { //SUCCESS
- S.value=0;
- return;
- }
- else
- { //UNSUCCESSFUL
- Put this process (PCB)
- In S → L(Q)&Block
- the process (sleep ())
- }
- }

- up (BSEMAPHORE S)
- {
- if (S.L(Q) is not empty)
- {
- Select a process from
- S.L(Q) &wakeup ();
- }
- Else
- S.value =1;
- }

#NOTE: The woken-up process will go to CS directly bcz it has already done comparison.
(0 Up 0), when Queue is not empty.



NOTE:

- As long as a process is in CS or atleast one process in the Block Queue, L then value of semaphore must be 0.
- If the value of semaphore is 0 then there may or may not be any block processes bcz at this situation it can be in CS.
- In Binary Semaphore one can never know no. of block processes by looking at Semaphore value.

- Semaphore is a general-purpose utility, used to solve varieties of problems.

Que1: Let $S=1$, then after performing the following operations {15P, 2V, 8P, 3V, 6P, 4V}.

The value of S in binary semaphore = ? Ans: 0 Size of (Q.L)= 19

Que2: In a certain client/server environment a database server is controlled with a counting S to meet its maximum concurrency requirement. The max concurrency level of the database server is 200 process request. What must be the initial value of counting S in this application.

Que3: In a certain application the following operations were completed on S in the given order {20P, 4V, 15P, 2V, 6P, 8V, 2P, 3V}. But the current value of $S= -10$. What could have been the initial value of S . Ans: $x=16$

Cases in Binary Semaphore

Case 1:

$S=1;$

P (S);

$S= 0$

Status= succ.

Case 2:

$S=0;$

P (S);

$S= 0$

Status=unsucc

Case 3:

$S=1;$

V (S);

$S= 1$

Status= succ

Case 4:

$S=0;$

V (S);

$S= 1$

Status= success

Queue is empty, only last process coming out from CS will make it

Successful in its attempt to get into CS



- A process can get blocked in either counting/binary semaphore if it performs: Down/UP? Down
- Up is always successful.

Que: //Concurrent independent processes

```
mutex=1;
```

```
Pri ; i= 1,9
```

```
while (1)
```

```
{
```

```
  P (mutex);
```

```
  <CS>
```

```
  V(mutex);
```

```
}
```

```
Pr10
```

```
while (1)
```

```
{
```

```
  V (mutex);
```

```
  <CS>
```

```
  V(mutex);
```

```
}
```

Que1: Maximum number of processes that can be in the CS? 10

Que2: Does this case ensure Mutual Exclusion or not ? No

Que:

S= 1, T= 0;

Mutex=1;

Pr_i;

while (1)

{

P (T);

print ('0');

print ('0');

V(S);

}

Pr_j;

while (1)

{

P (S);

print ('1');

print ('1');

V(T);

}

Que1: if both the process can run concurrently, what will be the regular expression for print statements? =Ans: $(1100)^+$

Que2: Does this case ensure Mutual Exclusion? ME, B.W, but not Progress

Que3: It is an example of which synchronization mechanism? strict alternation with Non-Busy waiting.

Que:

S= 1, T= 1;

Pr_i

while (1)

{

P (S);

P (T);

<CS>

V (T);

V (S);

}

Pr_j

while (1)

{

P (T);

P (S);

<CS>

V (S);

V (T);

}

Que 1: Does this case ensure Mutual Exclusion or not ? Yes? When?

Que 2: Progress? No

Que 3: Bounded waiting? No

Que 4: Is there any Deadlock potential? Yes

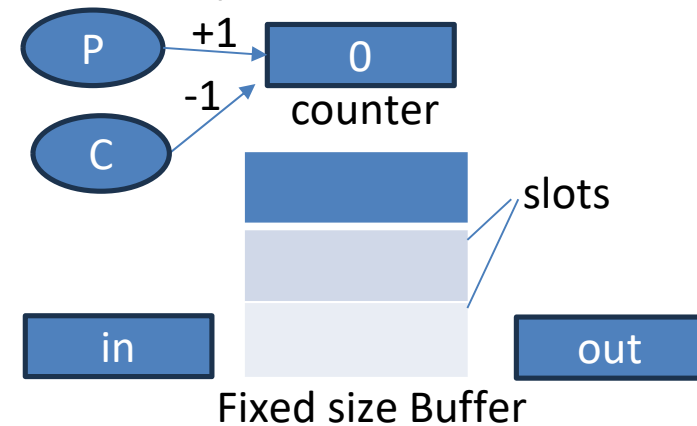
Que 5: What changes may be required to make it Dead Lock Free. OR in what order both processes should execute to avoid deadlock? Execute in same order.

- ☐ Mutual exclusion is a necessary but not sufficient condition for deadlock. If deadlock occurs, mutual exclusion must already be present along with other conditions like hold-and-wait, no preemption, and circular wait.

#Applications of Semaphores: Classical IPC problem

```
#define N
int Buffer [N]; //initialized
CSem Empty =N; //no.of empty slots in Buffer
CSem Full=0; //no. of Full slots in Buffer
BSem mutex =1; {used by P&C to access
                Buffer as CS→ allowing
                either P or C at a time}

Void producer (void)
{
    int itemp, in=0;
    While (1)
    {
        /* produce an item in temp */
        a) Produce item (itemp);
        b) Down (Empty);
        c) Down (mutex);
        d) Buffer [in]=itemp;
        e) in = (in + 1) %N
        f) Up (mutex);
        g) Up (full);
    }
}
```



```
Void consumer (void)
{
    int itemc, out=0;
    While (1)
    {
        a) Down (full);
        b) Down (mutex);
        c) itemc = Buffer [out];
        d) out = (out + 1) %N;
        e) Up (mutex);
        f) Up (Empty);
        g) process item (itemc);
    }
}
```

- Binary Semaphore: Co-operation
- Counting Semaphore :Competition

Does it ensure:

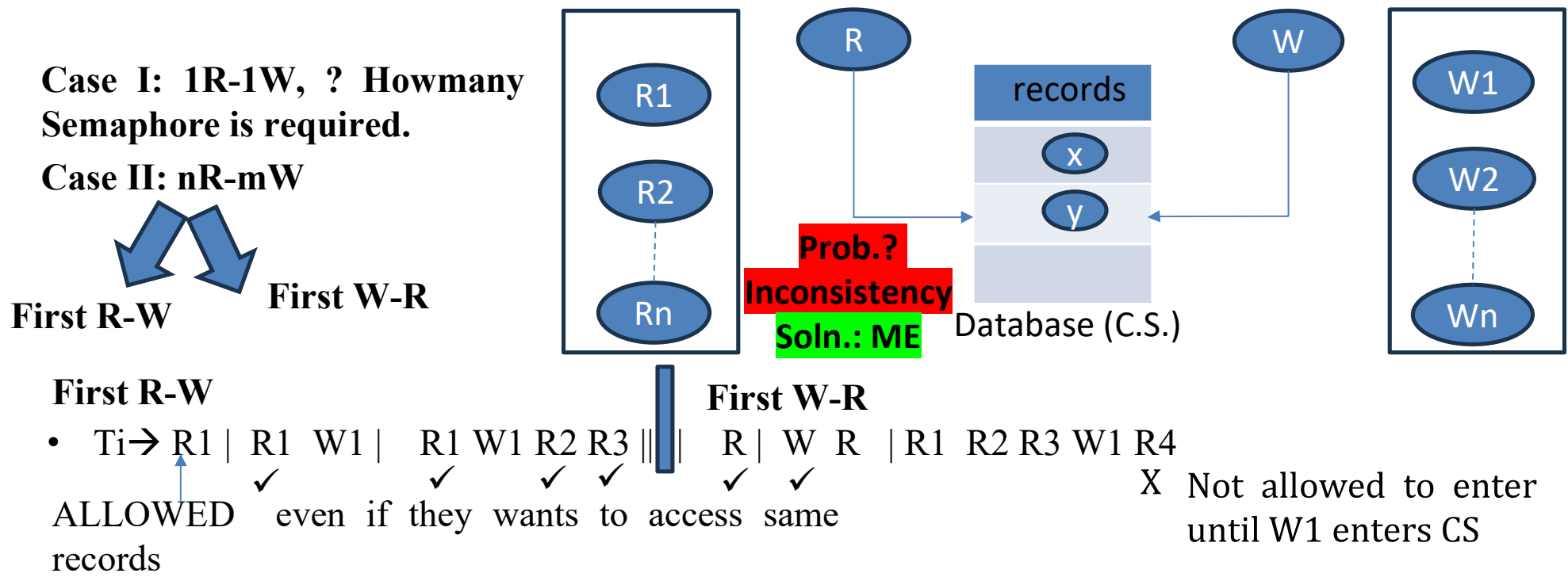
- ME -yes,
- BW- No
- Progress- No

Exp.

- P1 finished, sets turn = P2
 - P2 is not interested
 - P1 wants to enter again but **cannot**
- CS is free, but still blocked → Progress violated

Cont.

- **First R-W**: In this case **READERS** are always favored over **WRITERS**. As and when any reader is available, it is always allowed to enter Database and hence may causing the **problem of Starvation to Writers**.
- **First W-R**: In this case **WRITERS** are always favored over **READERS**. In the absence of Writer process, Readers are always allowed till a new writer becomes available.
- In the presence of a Writer, No New Reader process is allowed to enter Database till Writer is allowed to enter in it.
- The principle where in Writer process is waiting for the Readers to vacate the Database is known as “Turnstile”.



#Solution to First R-W using Semaphores

```
int rc=0 {readers count} // it becomes CS bw readers
    BSem Mutex =1; { Used to ensure mutual exclusion while updating rc.
                    Only one reader can modify rc at a time. }

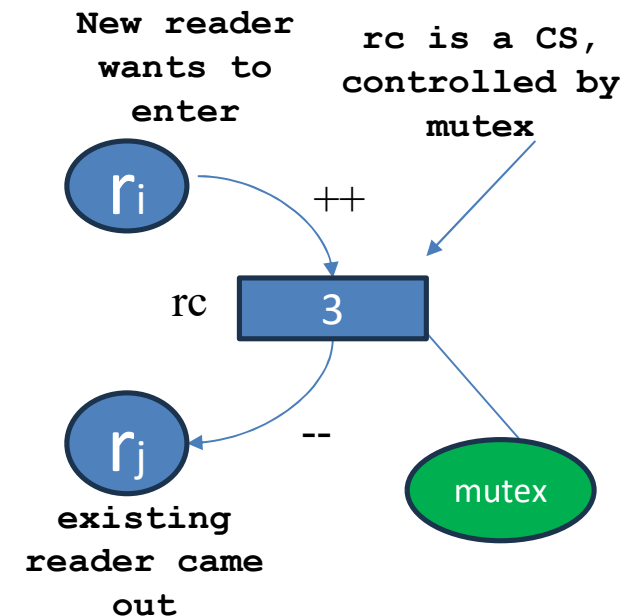
    BSem dB =1; { // =0, can ne done by writer, reader, readers }
                { used by Writers&Readers and Writers&Writers
                  to access dB as CS
                }
}
```

Solution:

```
Void writer(void)
{
    While (1)
    {
        a) Down (dB);
        b) <dB>;
        c) Up (dB);
    }
}
```

Up → Allows another writer OR Readers to enter

```
Void Reader (void)
{
    While (1)
    {
        a) Down (mutex); // Prevent race condition among readers
        b) rc=rc+1;
        c) If (rc==1) Down (dB) ; first reader locks it, This prevents writers
        d) Up (mutex);           from entering. .Other readers can directly
        e) <dB>                  enter
        f) Down (mutex);        // lock again to safely update rc
        g) rc=rc-1;
        h) if (rc==0) Up (dB) ; last reader releases it
        i) Up (mutex);
    }
}
```



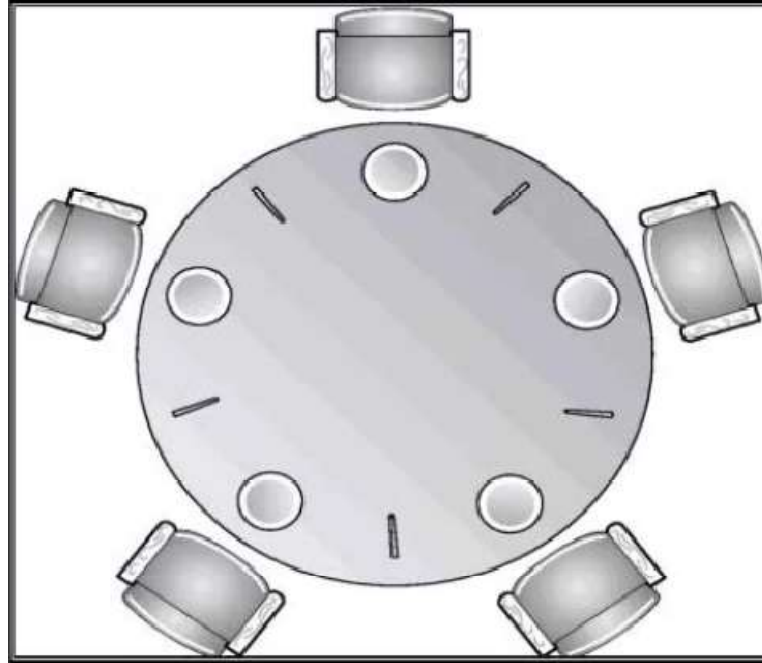
- ME → Yes (bw writer and Readers)
- Progress No
- Bounded Waiting- No

Note: After 1 writer no Reader, No writer is allowed, to maintain consistency.

Note: rc and dB both are CS

Que: When this will turn to 1R-1W from nR-W? Ans: removing rc==1

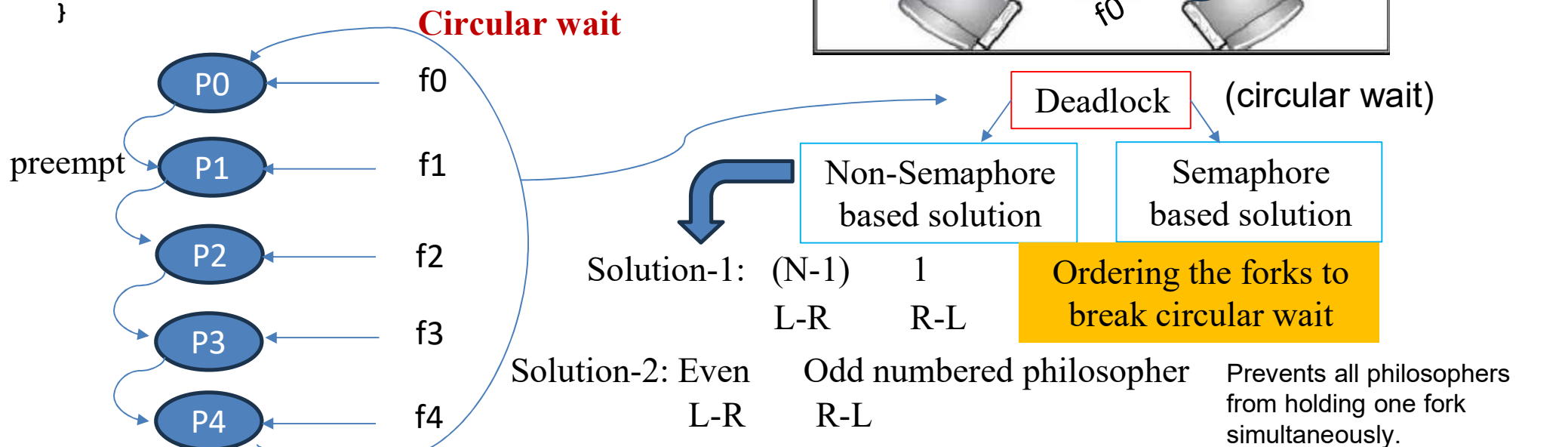
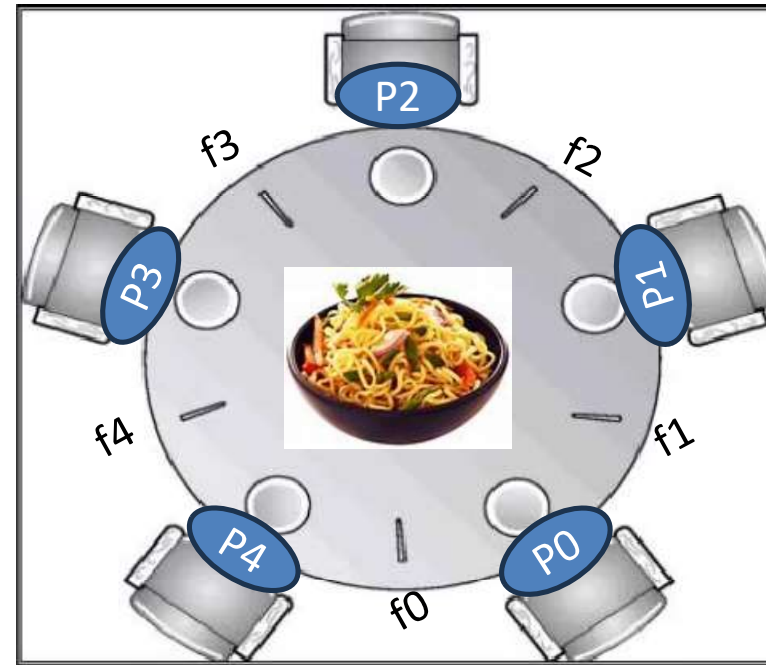
Remove reader concurrency and treat both reader and writer as **mutually exclusive processes**



Dining Philosopher Problem: ($N \geq 2$)

```

Void Philosopher (int i)
{
    While (1)
    {
        a) think (i);
        b) take-fork (i); //Left
        c) take-fork ((i+1)%N) //Right
        d) eat(i);
        e) put-fork (i);
        f) put-fork ((i+1)%N);
    }
}
    
```



Max. concurrency = 1, in both cases of Non-semaphore solutions.

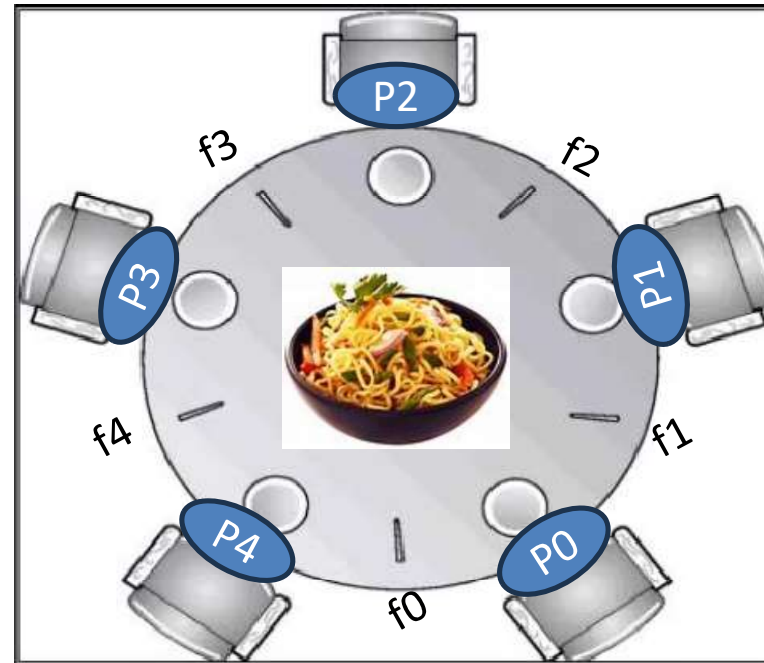
In general, '2'

3. How many extra fork to break the deadlock? --1

4. How many extra fork when all are allowed to eat concurrently..5

Dining Philosopher Problem: SEMAPHORE

```
#define N 5
#define THINK 1
#define HUNGRY 2
#define EATING 3
#define LEFT (i+N-1)%N //neighbor philosopher
#define RIGHT (i+1)%N
Int state[N] = {THINK}
Bsem mutex=1;
{
//Used by p's to access forks as CS}
  Bsem s[N]={0}; //s<-- for self
{
Used by P's for self test to go for eat/Not
}
```



Philosopher LOOP

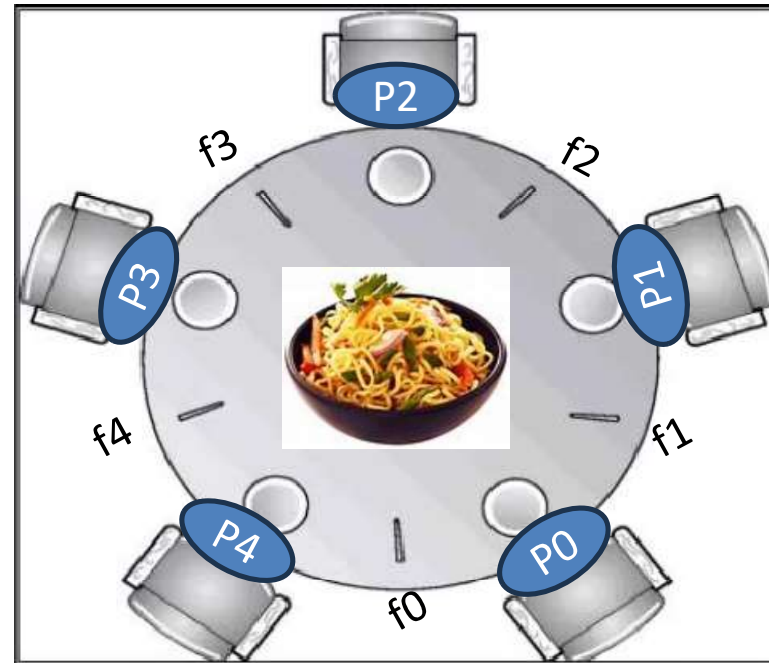
<pre>Void Philosopher (int i) { While (1) { a) think (i); b) take-forks (i); c) eat(i); d) put-forks (i); } }</pre>	<pre>Void take forks(int i) { a) Down (mutex); b) State [i]=HUNGRY; c) test(i); d) Up (mutex); e) Down (s[i]); }</pre>	<pre>Void test(int i) { if(state[i]==HUNGRY && state[LEFT]!=EATING && state[RIGHT]!=EATING) { State [i] =EATING Up (s[i]); } }</pre>	<pre>Void put forks (i) { a) Down (mutex); b) State [i]=THINK; c) test(LEFT); d) test(RIGHT); e) Up (mutex); }</pre>
---	--	--	--

Interchange will lead to deadlock

//S[i]→0=0, unsuccessful
1-successful in taking forks

Dining Philosopher Problem: SEMAPHORE

```
#define N 5
#define THINK 1
#define HUNGRY 2
#define EATING 3
#define LEFT (i+N-1)%N //philosopher
#define RIGHT (i+1)%N
Int state[N] = {THINK}
Bsem mutex=1;
{
//Used by p's to access forks as CS. Take
fork→ ++1, put fork→ --1}
Bsem s[N]={0}; //s<-- for self
{
Used by P's for self test to go for eat/Not
}
```



M.E → Guaranteed

Que: In Dining philosopher problem, there are X number of dinners and Y number of chopsticks. What is the minimum number of chopsticks required to ensure that there will be no deadlock?

- A) X
- B) $Y - X$
- C) $Y - X + 1$
- D) $\lceil Y/2 \rceil + 1$

Que: Dining Philosopher's problem is a

- 1.Producer - consumer problem
- 2.Classical IPC problem
- 3.Starvation problem
- 4.Synchronization primitive

Que: Let $m[0] \dots m[4]$ be mutexes (binary semaphores) and $P[0] \dots P[4]$ be processes. Suppose each process $P[i]$ executes the following:

```
wait (m[i]); wait (m[(i+1) mod 4]);  
.....  
release (m[i]); release (m[(i+1) mod 4]);
```

This could cause

- a) Deadlock
- b) Starvation but not Deadlock

Que: A solution to the Dining Philosophers Problem which avoids deadlock is:

- A) Ensure that all philosophers pick up the left fork before the right fork
- B) Ensure that all philosophers pick up the right fork before the left fork
- C) Ensure that one particular philosopher picks up the left fork before the right fork, and that all other philosophers pick up the right fork before the left fork
- D) If even numbered philosophers take their Left fork first then Right and odd numbered philosophers take their right fork first then Left.
- E) None of the above

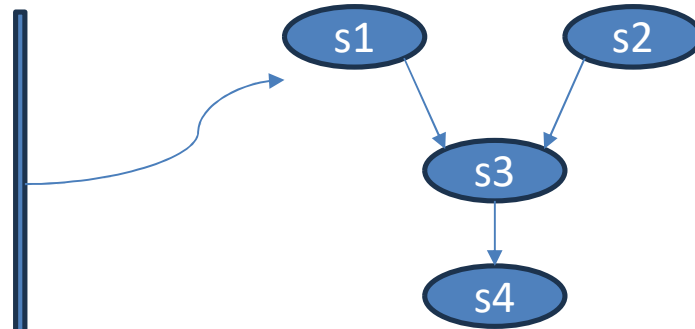
Concurrency Mechanism:

Concurrency: Def1: They should not have common variables.

Def2: The output of one should not serve as input to other.

Example1:

s1: $a = b + c$;
s2: $d = e * f$;
s3: $k = a + d$;
s4: $z = k / 10$;



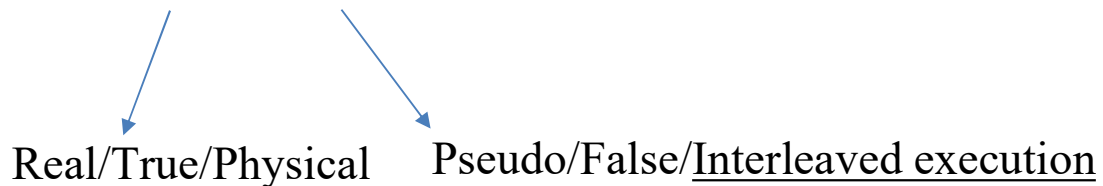
Precedence Graph

Block of statements

Example2:

s_i: $a = b + c$;
s_j: $d = b * c$;

Concurrency Types



Possible only when

→ Multi CPU

→ Multi core

→ Single CPU

With preemption

Benstein's conditions for concurrency:

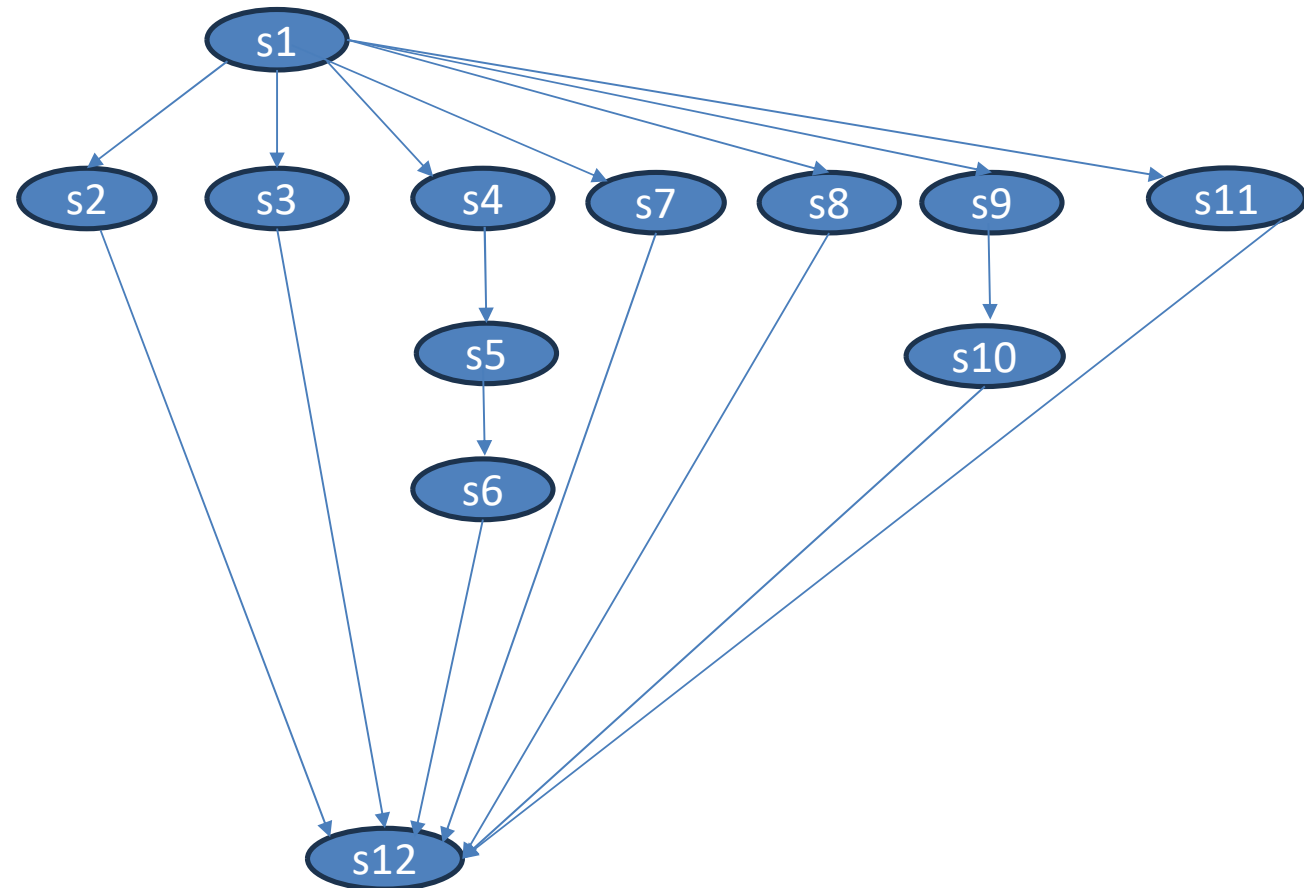
1. $R(s_i) \cap W(s_j) = \Phi$
2. $R(s_j) \cap W(s_i) = \Phi$
3. $W(s_i) \cap W(s_j) = \Phi$
4. $R(s_i) \cap R(s_j) = \text{may or may not be } \Phi$

Compulsory conditions

Parbegin-Parend/Cobegin-Coend: (Pascal, ALGOL, Ada)

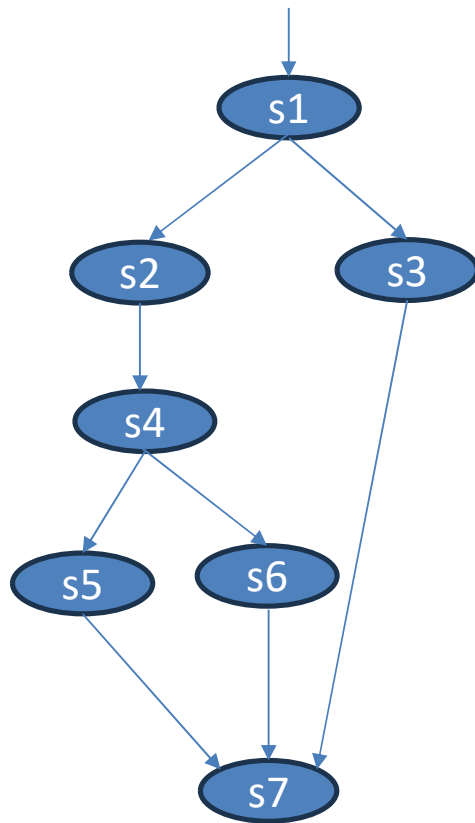
It is used in modeling precedence graph.

- s1;
- Parbegin
- s2, s3;
- begin s4, s5, s6 end
- s7
- s8
- begin s9, s10 end
- s11;
- Parend
- s12



#Precedence Constraints

Parbegin-Parend/Cobegin-Coend



- s1;
- Parbegin
- s3;
- begin
- s2;
- s4;
- Parbegin
- s5;
- s6;
- Parend;
- end;
- Parend;
- s7;

Curved arrows on the right side of the code sequence indicate the flow of execution: from s1 to s3, from s3 to s2, from s2 to s4, from s4 to s5 and s6, from s5 to s7, from s6 to s7, and from s3 to s7.

Parbegin-Parend/Cobegin-Coend

- Int x=0, y=0;

- Cobegin

- begin

- x=2;

- y=y+x;

- end

- begin

- y=5

- x=x+4;

- end;

- Coend;

Find the value of x and y?

x=4, y=5

- int x=0, y=22;

- Cobegin

- begin

- x=x+1;

- end

- begin

- x=y+1;

- end;

- Coend;

Find the value of x?

x=1 (if Thread 1 completes last)

If Thread 1 runs after Thread 2: x = 24

If Thread 2 runs last: x = 23

DEADLOCK

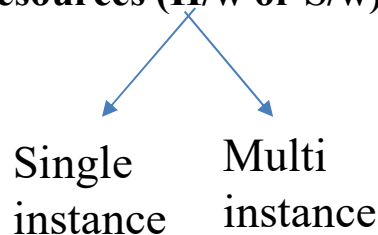
- **Concept**
- **System Model**
- **Necessary Condition**
- **Resource allocation graph**
- **Strategies for Deadlock Handling**

Concept: two or more processes are said to be in Deadlock 'if and only if' they wait for the happening of an event which would never happen.

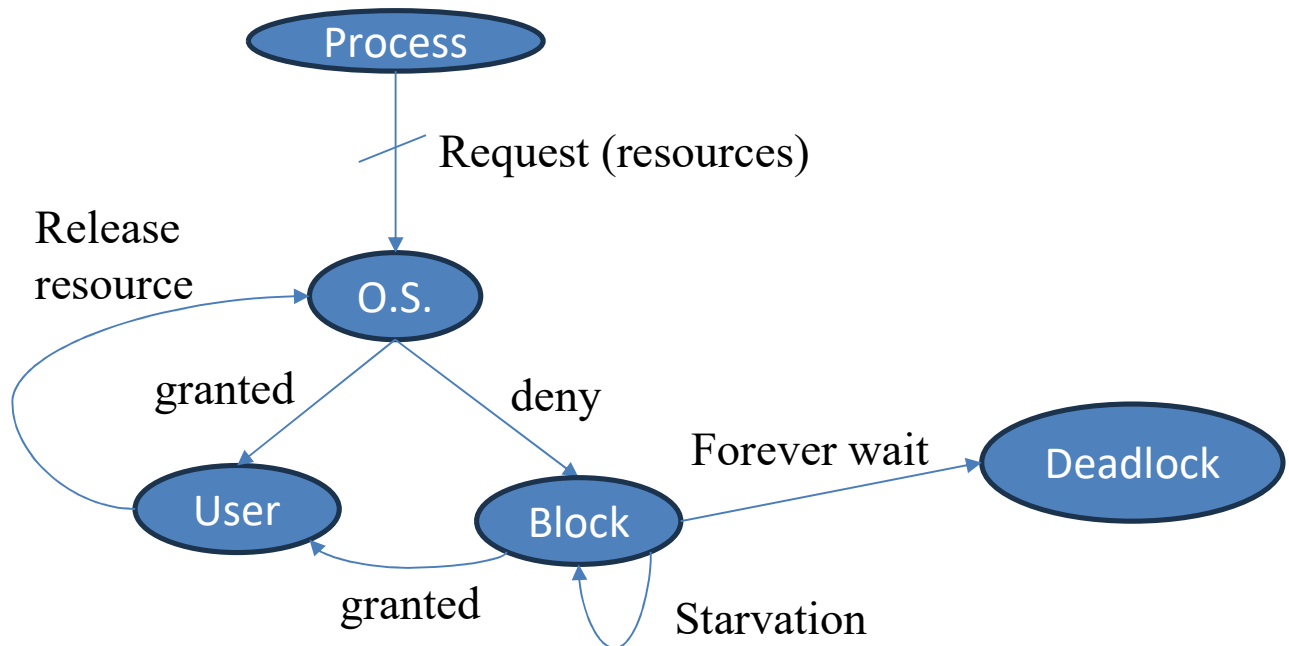
Deadlock is an undesirable feature as it leads to resource underutilization and reduce/ decline the efficiency of the system, drastically.

System Model:

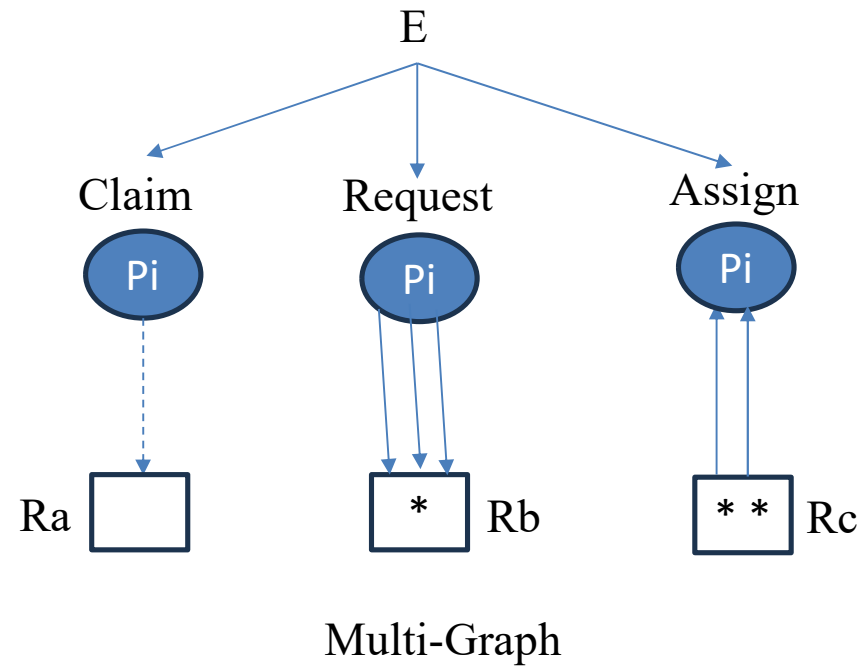
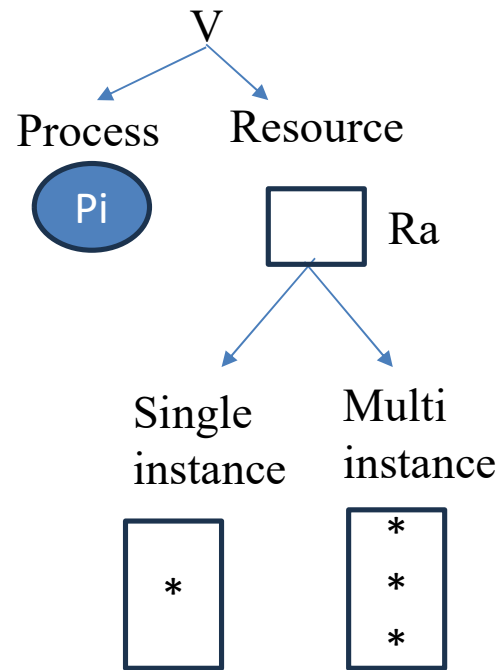
- 'n' processes ($P_1 \dots P_n$)
- 'm' resources (H/w or S/w)

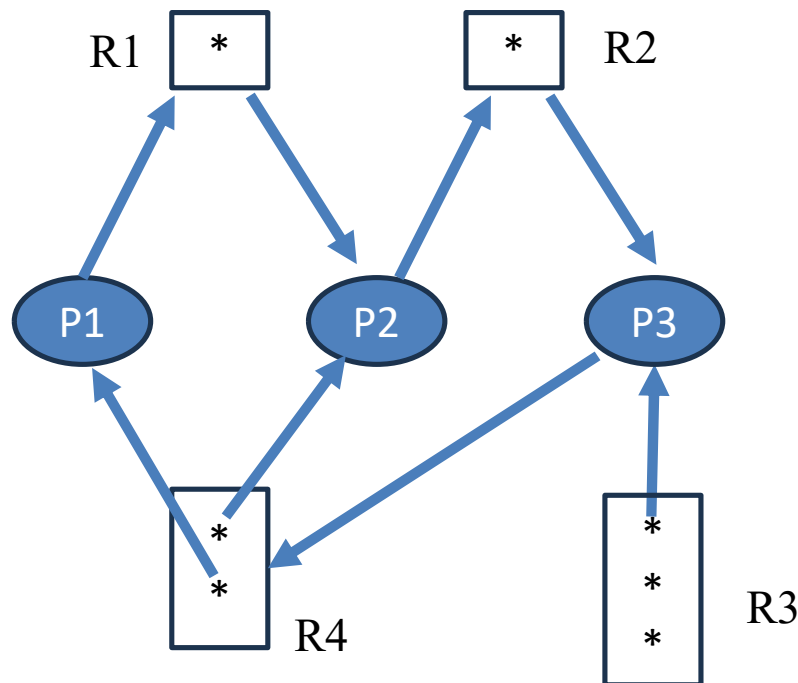


H/W resource: Memory,
S/W: semaphore, monitors, device drivers



→ Resource allocation graph: $G(V,E)$

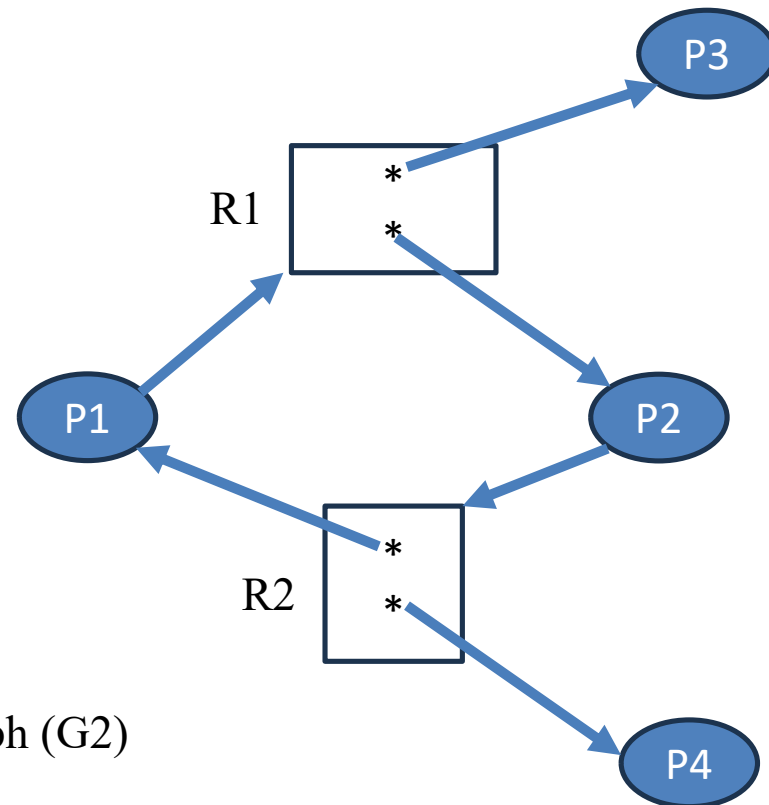




Graph (G1)

Cycle: P1, R1, P2, R2, P3, R4, P1

DEADLOCK



Graph (G2)

Here, p3 and P4 are not waiting for any resource so they might be running and will get completed. in another sense, completing of which will release the resources asked by the other processes. Cycle but no Deadlock

Note: 1. In case if system has resources of only single instance type, then cycle always implies deadlock (**necessary and sufficient condition**).

2. In case if system has resources of both single and multi-instance or only multi-instance then cycle is only necessary but not sufficient condition for deadlock.

Deadlock Handling Strategies

Deadlock

- | | | | |
|---------------|---------------------------|--|--|
| never occurs | 1. Prevention | //very restrictive | |
| | 2. Avoidance | //A prior knowledge (max.) | //Banker's Algorithm |
| D-lock occurs | 3. Detection And Recovery | Flexible i.e. no restriction on allowing | //Wait for Graph |
| | 4. Ignorance(No strategy) | deadlock | //Ostrich Algorithm
(window, linux, unix) |

2. Avoidance //Banker's Algorithm:-

a) **Safety Algo.** b) **Resource- request Algo.**

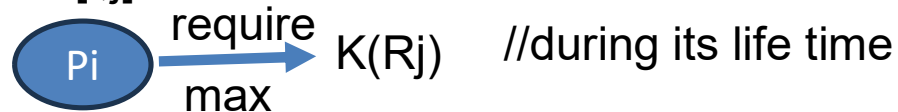
State of system;

→n=no. of processes;

→m= no. of resources;

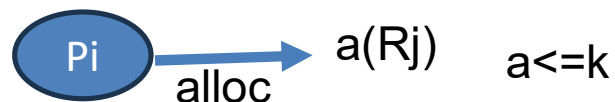
→**Maximum** $[1 \dots n, 1 \dots m]_{n \times m}$

Max[i,j] =k



→**Allocation** $[1 \dots n, 1 \dots m]_{n \times m}$

Alloc[l,j]=a



→**Need** $[1 \dots n, 1 \dots m]_{n \times m}$

need[i,j] =b



→ **Request** $[1 \dots n, 1 \dots m]_{n \times m}$

req[i,j] =c



→ Total [1...m]

Total[J]=z //there are a total of 'z' copies of R_J in system

→ Available [1...m]

Avail [J]=y //there are 'y' copies of R_J free avail @ time t;

Que: n=5, m =1 (R instance), $R_{\text{total resources}}=30$

Ans: [P2, P4, P5, P1, P3] Not unique

	Max R	Alloc R	Need R	Avail R
P1	10	5	5	3
P2	5	2	3	<5>
P3	25	10	15	<9>
P4	8	4	4	<15>
P5	12	6	6	<20>
				<30>

at last free

Note: system is said to be safe 'if and only if' the need of all the processes can be satisfied with the available resources in some order (safe sequence) otherwise the system is said to be unsafe.

Que:

Que: $n=5$, m (R instance)=3, Total available (10,5,7)

Ans: [P1, P3, P4, P0, P2] SAFE

	Max R	Alloc R	Need R	Avail R
P0	7,5,3	0,1,0		<>
P1	3,2,2	2,0,0		<>
P2	9,0,2	3,0,2		<>
P3	2,2,2	2,1,1		<>
P4	4,3,3	0,0,2		<>

Resource- request Algo.

1. $Req_i \leq Need_i$ // user

2. $Req_i \leq Avail$ // Bank

3. Assume to have satisfy the request

$Avail = Avail - Req_i$ $3L = 25L - 22L$

$Need_i = Need_i - Req_i$ $3L \leq 25L - 22L$

$Alloc_i = Alloc_i + Req_i$ User Account (passbook)

4. Run SAFETY Algo.

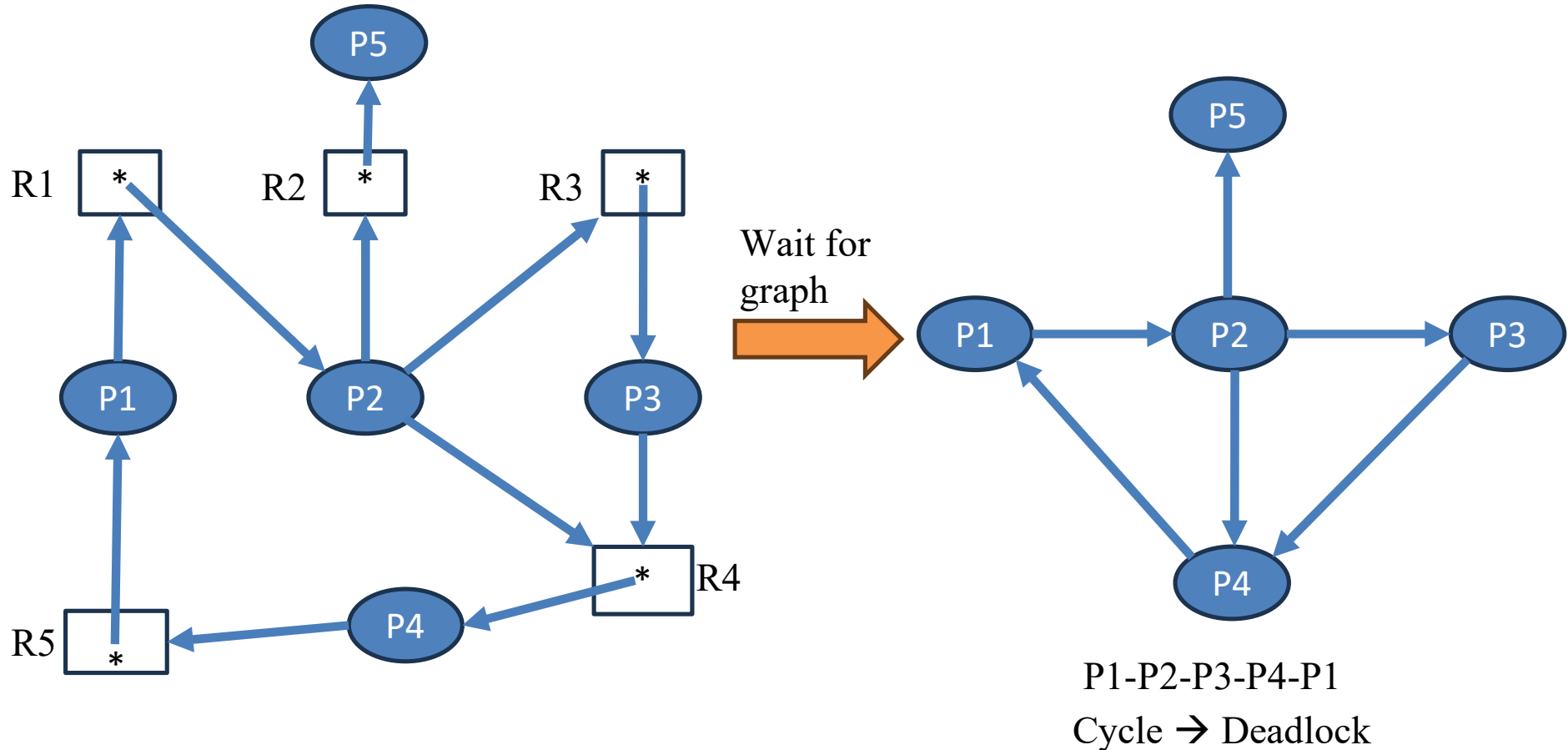
5. If the system is SAFE then GRANT; // and update

Else

Deny (Block);

Deadlock Detection and Recovery:

1. Detection for single instance of resource type:



Here P5 can run but this does not mean that there cannot be deadlock (OR) there is no Deadlock. Because we have to look for those who are willing to run onto CS. So, look only among them.
here P1 P2 P3 P4 are those processes and among them no one is able to complete their task so it is a deadlock.

Que:

2. Detection with Multi instance resources:

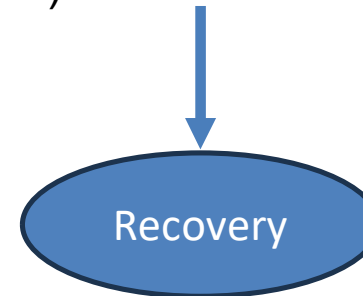
Que: $n=5$, m (R instance)=3, Total (A,B,C)= (7,2,6)

	Alloc A,B,C	Request A,B,C	Avail A,B,C
P0	0,1,0	0,0,0	0,0,0
P1	2,0,0	2,0,2	
P2	3,0,3	0,0,0	
P3	2,1,1	1,0,0	
P4	0,0,2	0,0,2	

$T0 = \langle P0, P2, P3, P4, P1 \rangle \rightarrow \text{SAFE}$

And if at Ts : P2 requests $\rightarrow \langle 0, 0, 1 \rangle$

$\langle P0, \dots \rangle \rightarrow \text{SATISFIES}$ But remaining
 $\rightarrow (P1, P2, P3, P4) \rightarrow \text{DEADLOCK}$



Note: system is said to be **SAFE** 'if and only if' the REQUEST of all the processes can be satisfied with the available resources in some order (safe sequence) otherwise the system is said to be in **DEADLOCK**. Those processes whose request are not satisfiable are said to be in **DEADLOCK**.

Recovery Strategies:

Suffer from
starvation problem

Process
termination

Resource
preemption

Kill All

Kill One

- No need to apply detection algo again.
 - All killed processes needs to restart.
 - Important (almost completed) processes also got killed.
- Need to apply detection algo again.
 - Important processes may be saved.

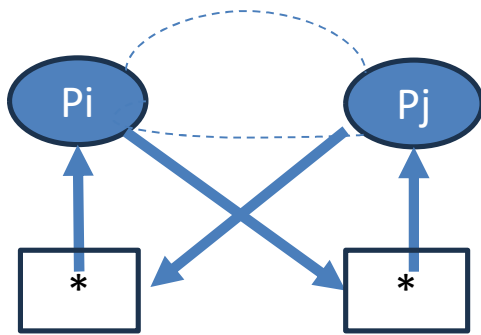
Deadlock PREVENTION:

Q.:When deadlock can arise

Ans:if all 4 necessary conditions hold simultaneously

Deadlock is PREVENTED by dissatisfying one or more of the **Necessary Conditions:**

1. Mutual Exclusion //not dissatisfiable [printer is inherently not sharable]
2. **!**(Hold &Wait): (Hold or Wait)



- a) Process must request all its resources prior to its commencement. **//Inefficient/ Starvation**
- b) Release all its resources before making a NEW/FRESH request. **//Starvation**

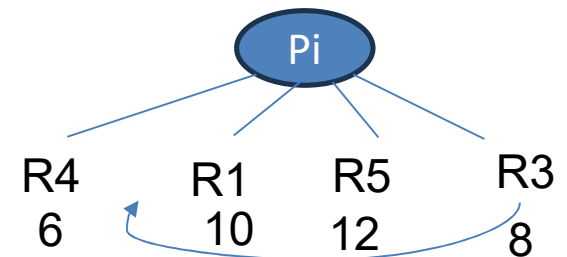
	Pid
R1	10
R2	12
R3	8
R4	6
R5	12
R6	9
R7	15
R8	18

3. **!**(NO Pre-emption):



4. Circular Wait: It is prevented by enforcing the following protocol:

- (1) Number all resources uniquely
- (2) Never allow a process to request a lower numbered resource than the last one requested/allocated.



Que:

Que1: Consider a system 'n' processes and a single resource 'R' having 6 copies. Each process needs two copies of 'R' to complete its execution.

--What is the minimum number of process in the system to cause a deadlock.

--What is the maximum number of processes to ensure a deadlock free system.

Min.=6

Max.=5

Que.2: $n=5$, min no. of resources $R=?$ To ensure Deadlock free system. Ans =24

	Max R
P1	5
P2	8
P3	2
P4	3
P5	10